Applications using Spring Cloud Function can be deployed in a few different ways. First of all, since they're still Spring Boot applications, you can package them as JAR artifacts or container images and deploy them on servers or container runtimes like Docker or Kubernetes, respectively, just like you did in the previous chapters.

Then, when Spring Native is included, you also have the option to compile them to native images and run them on servers or container runtimes. Thanks to instant startup time and reduced memory consumption, you can also seamlessly deploy such applications on serverless platforms. The next section covers how to use Knative to run your serverless workloads on Kubernetes.

Spring Cloud Function also supports deploying applications on vendor-specific FaaS platforms like AWS Lambda, Azure Functions, and Google Cloud Functions. Once you choose a platform, you can add the related adapter provided by the framework to accomplish the integration. Each adapter works in a slightly different way, depending on the specific platform and the configuration required to integrate the functions with the underlying infrastructure. The adapters provided by Spring Cloud Function don't require any changes to your business logic, but they might need some additional code to configure the integration.

When you use one of those adapters, you must choose which function to integrate with the platform. If there's only one function registered as a bean, that's the one used. If there are more (like in Quote Function), you need to use the `spring.cloud .function.definition` property to declare which function the FaaS platform will manage.

NOTE You can refer to the Spring Cloud Function official documentation for more details on the Spring Cloud Function adapters for AWS Lambda, Azure Functions, and Google Cloud Functions (https://spring.io/projects/spring-cloud-function).